

Proyecto: Gestión Inteligente de Trafico Urbano Experimentos Realizados y Resultados.

Marianne Coy y Daniel Diaz

Pontificia Universidad Javeriana

I. Introducción

El presente anexo describe los experimentos realizados sobre el sistema distribuido de gestión inteligente de tráfico urbano, con el objetivo de comparar el desempeño del diseño original del broker (modelo de un único hilo de procesamiento, denominado en adelante diseño simple) frente al diseño modificado para concurrencia (diseño multihilo), bajo las dos configuraciones de carga definidas en la Tabla 1 del enunciado del proyecto.

Las variables independientes consideradas fueron (i) el número de sensores generando información de la ciudad y (ii) el intervalo de tiempo entre mediciones. Las variables dependientes medidas fueron M1: cantidad de solicitudes almacenadas en la base de datos en un intervalo de 2 minutos, y M2: tiempo transcurrido desde que el usuario emite una acción de prioridad hasta que el semáforo refleja el cambio de estado en la base de datos.

II. Especificaciones de Hardware y Software

- **Hardware**

Las pruebas se ejecutaron sobre tres máquinas virtuales (VMs) administradas por la Pontificia Universidad Javeriana, accedidas de forma remota mediante escritorio remoto a través de VPN institucional. Cada VM tiene las siguientes características:

Parámetro	PC1 (Sensores/Broker)	PC2 (Analítica/Semáforos)	PC3 (Monitoreo/BD)
CPU	Intel Xeon Gold 6348 @ 2.60 GHz	Intel Xeon Gold 6348 @ 2.60 GHz	Intel Xeon Gold 6348 @ 2.60 GHz
Núcleos asignados	4 vCPU	4 vCPU	4 vCPU
RAM total	11 GB	11 GB	11 GB
SO	Ubuntu 22.04.5 LTS	Ubuntu 22.04.5 LTS	Ubuntu 22.04.5 LTS
Kernel	6.8.0-100-generic x86_64	6.8.0-100-generic x86_64	6.8.0-100-generic x86_64
Red	VPN institucional / red interna universitaria		

- **Software**

Componente	Version	Uso
Python	3.10.12	Lenguaje principal del sistema
pyzmq	27.1.0	Bindings ZeroMQ para Python
SQLite	3.37.2	BD principal (PC3) y replica (PC2)
ZeroMQ (libzmq)	Incluida en pyzmq 27.1.0	Middleware de mensajería distribuida

III. Experimentos Realizados

Se ejecutaron cuatro corridas independientes que cubren las cuatro celdas de la matriz experimental: el diseño simple y el multihilo bajo cada uno de los dos niveles de carga. La cuadrícula simulada es de 3×3 intersecciones (9 intersecciones, cada una con tres sensores: cámara, espira inductiva y GPS), lo que produce 27 sensores activos en el escenario de baja carga y 54 sensores en el escenario de alta carga (dos grupos independientes de sensores ejecutándose en paralelo).

- **Métrica 1 — Throughput de almacenamiento**

Para cada corrida se permitió la estabilización del sistema durante 30 s y posteriormente se midió, durante una ventana de 120 s, la cantidad de eventos persistidos en la tabla eventos_sensores de la base de datos principal en PC3. La consulta SQL utilizada filtra por la columna recibido_en para considerar únicamente eventos del intervalo de medición.

- **Métrica 2 — Latencia extremo a extremo**

Para cada corrida se realizaron 5 repeticiones consecutivas (separadas por 3 s) de la siguiente operación: un cliente envía la acción PRIORIDAD_EMERGENCIA para la intersección INT_C3 mediante el patrón REQ/REP de ZeroMQ contra el servicio de monitoreo en PC3. Se registra el instante t_0 inmediatamente antes del envío y, mediante polling de 50 ms sobre la base de datos, se detecta el instante t_1 en el cual la intersección queda con estado VERDE. La latencia reportada es $M2 = t_1 - t_0$, e incluye el tiempo de red REQ/REP, el procesamiento en analítica y el comando hasta el controlador de semáforos.

IV. Resultados

La Tabla 1 consolida los resultados de las cuatro corridas siguiendo el formato exigido por el enunciado.

Configuración de carga	Diseño simple	Diseño multihilo
1 sensor de cada tipo, datos cada 10 s	M1 = 324 solicitudes M2 = 30,0 ms (avg)	M1 = 305 solicitudes M2 = 29,7 ms (avg)
2 sensores de cada tipo, datos cada 5 s	M1 = 1.216 solicitudes M2 = 53,5 ms (avg)	M1 = 1.212 solicitudes M2 = 39,0 ms (avg)

Tabla 1. Resultados de los escenarios de prueba.

Adicionalmente, la Tabla 2 presenta el detalle estadístico de la métrica de latencia (mínimo, promedio y máximo) y la tasa de eventos por segundo derivada de M1.

Corrida	M1 (evt/2 min)	Tasa (evt/s)	M2 avg (ms)	M2 [mín-máx] (ms)
Simple, Carga A	324	2,70	30,0	17,2 – 69,9
Multihilo, Carga A	305	2,54	29,7	14,0 – 71,3
Simple, Carga B	1.216	10,13	53,5	16,2 – 83,8
Multihilo, Carga B	1.212	10,10	39,0	18,5 – 70,7

Tabla 2. Detalle estadístico de las mediciones.

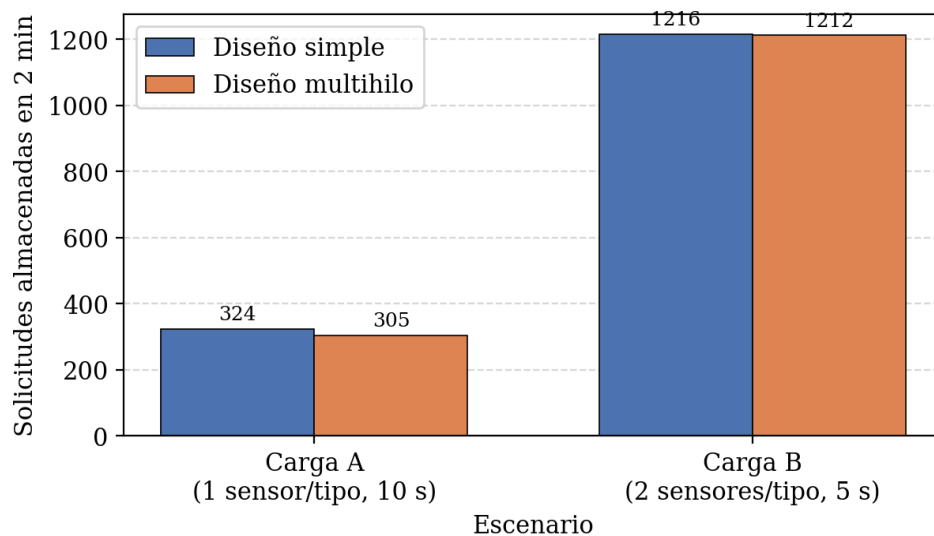


Figura 1. M1 — Solicitudes almacenadas en la base de datos durante 2 minutos.

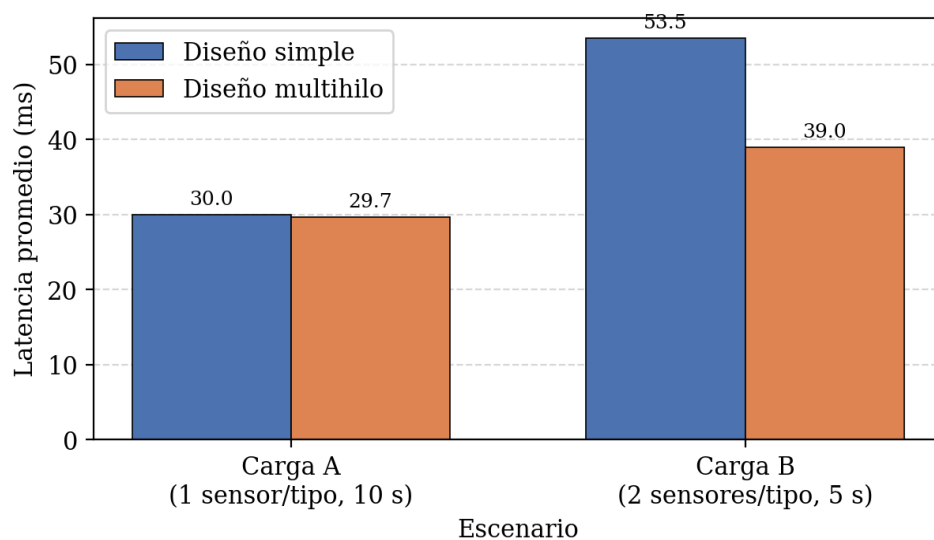


Figura 2. M2 — Latencia promedio (en milisegundos) desde la solicitud del usuario hasta el cambio de semáforo en la base de datos.

V. Análisis

• Comportamiento de la métrica M1 (throughput)

Los resultados de la Figura 1 y la Tabla 1 muestran que el throughput escala de forma aproximadamente lineal con la carga inyectada por los sensores: al duplicar el número de sensores y reducir el intervalo a la mitad, el número de eventos almacenados se incrementa en un factor cercano a 4 (de ~315 eventos en el escenario de baja carga a ~1.214 en el escenario de alta carga), lo cual coincide con la tasa teórica esperada ($54 \text{ sensores} \times 24 \text{ ciclos} = 1.296 \text{ eventos en 2 minutos}$). La pequeña diferencia respecto al valor teórico (~94 % del esperado) puede atribuirse al tiempo de estabilización inicial dentro de la ventana de medición y a la jitter en los temporizadores de los sensores.

Comparando los dos diseños bajo la misma carga, se observa que el broker multihilo no produce una ganancia significativa en throughput: en el escenario de baja carga incluso se observa una ligera disminución del 5,9 % (305 vs. 324), mientras que en el escenario de alta carga la diferencia es prácticamente nula (-0,3 %, dentro del margen de error estadístico). Este resultado es consistente con la naturaleza del patrón ZeroMQ PUSH/PULL utilizado entre los sensores y el broker: dicho patrón implementa ya internamente una cola eficiente de bajo costo, lo que deja al broker simple lejos de su límite de capacidad para la carga evaluada (alrededor de 10 evt/s).

• Comportamiento de la métrica M2 (latencia)

A diferencia de lo observado en M1, la métrica de latencia (Figura 2) sí presenta una diferencia relevante entre los dos diseños en condiciones de alta carga: el diseño multihilo reduce la latencia promedio en aproximadamente un 27 %, pasando de 53,5 ms en el diseño simple a 39,0 ms en el multihilo. En el escenario de baja carga, en cambio, las latencias son estadísticamente equivalentes (30,0 vs. 29,7 ms), confirmando que la ventaja del paralelismo sólo se manifiesta cuando el sistema está bajo presión.

La explicación técnica es coherente con el diseño implementado: bajo carga alta, el broker simple procesa todos los mensajes de forma secuencial, encolándolos en el socket PUB antes de que la analítica pueda consumirlos, lo que retrasa la propagación de comandos prioritarios. En el diseño multihilo, en cambio, el pool de workers mantiene varias rutas de procesamiento en paralelo, de modo que un comando de prioridad puede ser procesado por un worker mientras los otros siguen atendiendo el flujo de eventos de sensores. Aunque el throughput agregado se mantiene similar (el cuello de botella se desplaza hacia la persistencia en SQLite y el procesamiento de la analítica), la latencia individual de las acciones de control mejora visiblemente.

• Discusión integral

Estos hallazgos permiten extraer dos conclusiones importantes. En primer lugar, el throughput global del sistema está limitado más por la persistencia y el procesamiento de la analítica que por el broker, por lo que las dos versiones del broker resultan equivalentes en términos de cantidad de eventos almacenados. En segundo lugar, sin embargo, el diseño multihilo aporta una mejora medible en la latencia de respuesta del sistema bajo carga alta, que es precisamente el escenario donde un sistema de control de tráfico real más necesita garantías de responsiveness. Bajo carga baja, ambos diseños

son indistinguibles, y el multihilo introduce un sobrecosto pequeño (lock sobre el socket PUB compartido) que no se traduce en una mejora visible.

VI. Conclusiones

- El throughput del sistema escala de forma cercana a la lineal con la carga inyectada por los sensores, alcanzando aproximadamente el 94 % del máximo teórico en el escenario de mayor carga.
- Los dos diseños presentan throughput equivalente en las dos cargas evaluadas, ya que el broker no constituye el cuello de botella del sistema bajo estas condiciones.
- El diseño multihilo reduce la latencia extremo a extremo en aproximadamente un 27 % bajo carga alta (de 53,5 ms a 39,0 ms), aportando un beneficio claro en la respuesta a comandos de prioridad cuando el sistema está bajo presión.
- Bajo carga baja, los dos diseños son indistinguibles en ambas métricas, lo que indica que el sobrecosto de la sincronización del multihilo es despreciable.
- La latencia extremo a extremo se mantiene por debajo de los 60 ms en todos los escenarios evaluados, lo cual es ampliamente suficiente para una aplicación de control de tráfico en tiempo real.